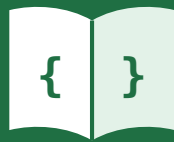

Module 07 — Spring Data MongoDB

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 The Document Model	2
2.2 Embed or Reference?	2
2.3 MongoDB Settings in Spring Boot 4	3
3. Step-by-Step Examples	3
3.1 Document Modelling	3
3.2 Repository Queries	4
3.3 MongoTemplate: Dynamic Queries and Atomic Updates	5
3.4 The Aggregation Pipeline	6
3.5 Indexes	7
3.6 Multi-Document Transactions	7
3.7 Testing with Testcontainers	8
4. Common Mistakes and Best Practices	9
5. Summary	9
6. Further Reading	10

1. Learning Goals

By the end of this module you will be able to:

- Model data as documents, and choose between embedding and referencing
- Write derived and JSON queries with `MongoRepository`
- Run dynamic queries and atomic updates with `MongoTemplate`
- Compute reports inside the database with an aggregation pipeline
- Define unique and text indexes
- Use multi-document transactions on a replica set
- Test MongoDB code with `Testcontainers`

Prerequisites: Modules 05–06 (Spring Data, transactions, `Testcontainers`) · **Estimated time:** 5 hours · **Docker required**

2. Concepts

2.1 The Document Model

MongoDB stores data not in tables but in JSON-like **documents** (BSON). A document can contain nested objects and arrays, and documents in the same collection may have different fields.

Relational (PostgreSQL)	MongoDB
Table	Collection
Row	Document
Column	Field
JOIN	Embedding or <code>\$lookup</code> / references
Schema (DDL)	Flexible. The application and indexes enforce the rules

2.2 Embed or Reference?

Embed	Reference
Read together (a book and its reviews)	Read or changed independently (a publisher)
Bounded in number	Can grow without limit

Embed	Reference
Meaningless on its own	Shared by many documents

i Note

A document can be at most 16 MB. Do not embed lists that grow without limit (e.g. all sales of a book). Put them into a separate collection.

2.3 MongoDB Settings in Spring Boot 4

Connection settings live under `spring.mongodb.*`. Old keys such as `spring.data.mongodb.uri` are deprecated. Settings specific to Spring Data stay under `spring.data.mongodb.*`.

3. Step-by-Step Examples

With Docker running, start the application. MongoDB starts automatically as a replica set:

```
./mvnw -pl modules/07-data-mongodb/lesson -am spring-boot:run
```

MongoDB has no migrations. The tour inserts sample data itself when the collection is empty. The settings:

```
mongodb:
  database: bookstore                                # Boot 4: connection settings live under
  ↪ spring.mongodb.*
data:
  mongodb:
    auto-index-creation: true                        # create @Indexed / @TextIndexed indexes at
    ↪ startup
    representation:
      big-decimal: decimal128                        # exact decimals (the current default) – stated
    ↪ explicitly
```

3.1 Document Modelling

Goal: store a book as one document, with embedded reviews, a referenced publisher and free-form attributes.

```

@Document("books")
public record Book(
    @Id @Nullable String id,
    @Indexed(unique = true) String isbn,           // Lesson 3.5: a unique index
    @TextIndexed String title,                   // Lesson 3.5: part of the
        ↪ full-text index
    List<String> authors,                         // arrays are first-class in
        ↪ MongoDB
    BigDecimal price,                           // stored as Decimal128 (see
        ↪ application.yaml)
    int stock,
    List<String> categories,
    Map<String, String> attributes,               // different books, different
        ↪ attributes – no schema change
    @DocumentReference @Nullable Publisher publisher, // stores only the
        ↪ publisher's id
    List<Review> reviews) {                     // embedded: stored inside
        ↪ this document

```

Expected output (the raw document in the database):

```

== 3.1 A document
{"isbn": "9780134685991", "title": "Effective Java", "authors": ["Joshua Bloch"],
 "price": {"$numberDecimal": "89.90"}, "stock": 5, "categories": ["java",
 ↪ "best-practices"],
 "attributes": {"language": "en", "pages": "412"}, "publisher": {"$oid": "6ab4..."},
 "reviews": [{"author": "ayse", "stars": 5, "text": "Harika"}, ...]}

```

The reviews are inside the document. For the publisher only the id is stored, and Spring Data resolves it when reading.

Its test: `catalog/BookRepositoryTest.reviewsAreEmbeddedAndThePublisherIsReferenced`

i Note

BigDecimal values are stored as the exact decimal type Decimal128, which is the default in current Spring Data MongoDB. We still state it explicitly with `representation.big-decimal: decimal128`. Older versions stored BigDecimal as a **string**, in which case `$sum/$avg` do not work and sorting is alphabetical ("100" < "9").

3.2 Repository Queries

Goal: use the repository model known from JPA with MongoDB.

```
public interface BookRepository extends MongoRepository<Book, String> {

    Optional<Book> findByIsbn(String isbn);

    List<Book> findByAuthorsContaining(String author);           // matches one
    ↪ element of the array

    List<Book> findByPriceLessThanOrderByPrice(BigDecimal max);

    @Query("{ 'attributes.language': ?0 }")                     // a MongoDB JSON
    ↪ query
    List<Book> findByLanguage(String language);
}
```

Expected output:

```
== 3.2 Repository queries
by author:    [Effective Java, Java Puzzlers]
by language:  [Kürk Mantolu Madonna]
publisher:    Publisher[id=6ab4..., name=Addison-Wesley, country=US]
```

3.3 MongoTemplate: Dynamic Queries and Atomic Updates

Goal: build queries from variable criteria, and update a document in one step without reading it.

```
public List<Book> search(@Nullable String category, @Nullable BigDecimal maxPrice) {
    List<Criteria> parts = new ArrayList<>();
    if (category != null) {
        parts.add(where("categories").is(category));    // "is" on an array field =
    ↪ "contains"
    }
    if (maxPrice != null) {
        parts.add(where("price").lte(maxPrice));
    }
    Query query = parts.isEmpty() ? new Query() : new Query(new
    ↪ Criteria().andOperator(parts));
    return mongo.find(query.with(Sort.by("price")), Book.class);
}
```

Taking from stock performs the “is there enough stock?” check and the change in **one atomic operation**. Two concurrent requests cannot push the stock below zero:

```

public boolean takeFromStock(String isbn, int quantity) {
    var onlyIfEnough = new Query(where("isbn").is(isbn).and("stock").gte(quantity));
    var result = mongo.updateFirst(onlyIfEnough, new Update().inc("stock", -quantity),
        ↪ Book.class);
    return result.getModifiedCount() == 1;           // check and change in ONE atomic
        ↪ operation
}

public void addReview(String isbn, Review review) {
    mongo.updateFirst(new Query(where("isbn").is(isbn)), new Update().push("reviews",
        ↪ review), Book.class);
}

```

Expected output:

```

== 3.3 MongoTemplate
java ≤ 90:    [Java Puzzlers, Effective Java]
take 1 copy: true, stock now 9

```

3.4 The Aggregation Pipeline

Goal: compute the number of books and the average price per category inside the database.

A pipeline passes documents through stages in order: \$unwind turns an array into rows, \$group groups, \$project shapes and \$sort sorts:

```

public List<CategoryStats> statsPerCategory() {
    var pipeline = newAggregation(
        unwind("categories"),           // one row per
        ↪ (book, category)
        group("categories").count().as("count").avg("price").as("averagePrice"),
        project("count")
            .and("_id").as("category")

        ↪ .and(ArithmeticOperators.Round.roundValueOf("averagePrice").place(2)).as("averagePrice")
        sort(Sort.by(Sort.Order.desc("count"), Sort.Order.asc("category"))));
    return mongo.aggregate(pipeline, Book.class, CategoryStats.class).getMappedResults();
}

```

Expected output:

```

== 3.4 Aggregation
CategoryStats[category=java, count=3, averagePrice=79.97]

```

```
CategoryStats[category=best-practices, count=1, averagePrice=89.90]
CategoryStats[category=roman, count=1, averagePrice=45.00]
CategoryStats[category=spring, count=1, averagePrice=95.00]
```

3.5 Indexes

Goal: speed up queries and protect rules (a unique ISBN) inside the database.

With `auto-index-creation: true`, the `@Indexed(unique = true)` and `@TextIndexed` annotations (section 3.1) become indexes at startup. A second book with the same ISBN throws a `DuplicateKeyException`. The text index enables full-text search that also recognises word stems:

```
public List<Book> fullText(String words) {
    return
        ↪ mongo.find(TextQuery.queryText(TextCriteria.forDefaultLanguage()).matchingAny(words)),
        ↪ Book.class);
}
```

Expected output:

```
== 3.5 Text index
"puzzlers" → [Java Puzzlers]
```

Its test: `catalog/CatalogOperationsTest`

★ Tip

In production, manage indexes deliberately with `mongo.indexOps(...)` or a migration tool rather than at application startup, which can take a long time on large collections.

3.6 Multi-Document Transactions

Goal: store an order and decrease the stock, in two different collections, all or nothing.

Spring Boot registers no transaction manager for MongoDB. We define it ourselves:

```
@Configuration(proxyBeanMethods = false)
public class MongoTransactionConfiguration {

    @Bean
    MongoTransactionManager transactionManager(MongoDatabaseFactory databaseFactory) {
        return new MongoTransactionManager(databaseFactory);
    }
}
```

```

@Service
public class OrderService {

    private final MongoTemplate mongo;

    public OrderService(MongoTemplate mongo) {
        this.mongo = mongo;
    }

    @Transactional // uses the
    ↪ MongoTransactionManager bean
    public void place(String customer, String isbn, int quantity) {
        mongo.insert(new Order(null, customer, isbn, quantity, Instant.now())); // 1st
        ↪ document ...

        Inventory inventory = mongo.findById(isbn, Inventory.class);
        if (inventory == null || inventory.quantity() < quantity) {
            throw new IllegalStateException("Not enough stock for " + isbn); // ... is
            ↪ rolled back here
        }
        mongo.save(new Inventory(isbn, inventory.quantity() - quantity)); // 2nd
        ↪ document
    }
}

```

Expected output:

```

== 3.6 Transaction
rolled back: Not enough stock for 9781617297571 – orders stored: 0

```

Its test: orders/OrderServiceTest. It fails when the transaction manager bean is removed, because the order document stays behind.

! Important

MongoDB transactions work only on a **replica set** (or a sharded cluster). `compose.yml` starts a single-node replica set (rs0). In tests, `withReplicaSet()` provides it (section 3.7).

3.7 Testing with Testcontainers

```

@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

```

```
static final MongoDBContainer MONGO = new MongoDBContainer("mongo:8.0.32") // same
↳ image as compose.yaml
    .withReplicaSet(); // Testcontainers 2: opt-in;
    ↳ needed for transactions

@Bean
@ServiceConnection
MongoDBContainer mongoContainer() { // not "mongo": Boot already has a
↳ MongoClient bean with that name
    return MONGO;
}
}
```

@DataMongoTest loads only the MongoDB infrastructure and the repositories. Unlike with JPA, there is **no rollback** after each test. That is why the tests delete and re-insert their own data in @BeforeEach.

! Warning

Do not name the @Bean method that returns the container mongo. Boot's MongoClient bean already has that name, and the beans clash.

4. Common Mistakes and Best Practices

!! Caution

“MongoDB is schemaless” is no reason to give up data quality. Protect the rules with unique indexes, application validation and, if needed, MongoDB's JSON Schema validation.

- **Do:** model documents by **how they are read**. Embed what is read together.
- **Don't:** turn relational tables one-to-one into collections and use references everywhere. Every read becomes several queries.
- **Do:** use atomic updates such as \$inc, \$push and \$set instead of “read, modify, write”.
- **Don't:** store values such as prices as double or strings. Use Decimal128.
- **Do:** index frequently queried fields and check with explain() that the query uses the index.

5. Summary

- MongoDB stores data in flexible documents. Embedding and referencing are chosen by how data is read.

- `MongoRepository` offers derived and JSON queries. `MongoTemplate` is for dynamic queries and atomic updates.
- The aggregation pipeline computes reports inside the database.
- `@Indexed(unique = true)` and `@TextIndexed` define indexes.
- Multi-document transactions need a replica set and a `MongoTransactionManager` bean.
- In Boot 4, connection settings live under `spring.mongodb.*`. Store `BigDecimal` as `decimal128`.

Next step: [Exercises](#)

6. Further Reading

- [Spring Data MongoDB Reference](#)
- [Template Query Operations](#) · [Aggregation Framework](#)
- [Document References](#) · [Index Management](#)
- [Sessions & Transactions](#)
- [Spring Boot — NoSQL \(MongoDB\)](#)
- [MongoDB — Data Modeling](#)